INTERNSHIP PROJECT REPORT

API Monitoring & Alert System

Scheduled health checks, downtime detection, incident tracking and real-time alerts

Submitted by	SINCHANA RAVI GOWDA
Internship	SPARKIIT - Full-Stack Development
Internship period	5 October 2026 - 20 November 2026
Technologies	Node.js, Express, SQLite, node-cron, WebSocket, Chart.js
Source code	https://github.com/SinchanaRG26/sparkit_internship/tree/main/project1-api-monitor
Report date	1 November 2026

Contents

- 1. Abstract and Introduction
- 2. Objectives
- 3. Methodology / Implementation
- 4. Source Code and Technical Details
- 5. Output Screenshots
- 6. Results and Analysis
- 7. Conclusion
- 8. References

1. Abstract and Introduction

Abstract

Modern applications depend on web APIs. When an API becomes slow or stops responding, every application that relies on it is affected, often before anyone notices. This project implements an **API Monitoring & Alert System** that automatically checks the health of web APIs on a schedule, measures their response time, detects downtime, records incidents, and notifies the user the moment something goes wrong.

The backend is built with Node.js and Express. A cron-based scheduler issues HTTP requests to each monitored endpoint, measures latency and stores every result in a SQLite database. A downtime-detection algorithm declares an endpoint DOWN after two consecutive failed checks and opens an incident; the incident is closed automatically when the endpoint recovers. Alerts (DOWN, RECOVERED and SLOW) are shown on the dashboard, written to a log file and can be delivered through Telegram, e-mail or a Slack/Discord webhook. A web dashboard, kept up to date through WebSockets, presents live status cards, uptime bars, a response-time chart, a performance report (uptime, average, p95, minimum and maximum) and the incident history.

Introduction

An **API endpoint** is a web address that applications call to obtain data or perform an action. **Monitoring** means repeatedly checking that the endpoint is reachable, returns a successful response and answers fast enough. Commercial services such as UptimeRobot, Pingdom and Better Stack provide this capability; this project reproduces the core of such a service in order to understand how it works.

The assignment focuses on three concepts, each of which is implemented in the project:

- **Cron jobs** - a scheduler wakes up every 5 seconds and runs the checks that are due; an hourly job removes old data.
- **Logs** - every event (start-up, failed check, alert, outage) is written with a timestamp to `data/monitor.log` and shown on the dashboard.
- **Notifications** - alerts are created for downtime, recovery and slow responses and are pushed to the dashboard and to optional external channels.

2. Objectives

The objectives of the project, taken from the internship brief and extended with additional features, are:

#	Objective	How it is met
1	Monitor API response times	Every check measures latency in milliseconds; results are stored and charted.
2	Detect downtime	An endpoint becomes DOWN after 2 consecutive failures (non-2xx status, timeout or network error) and recovery is detected automatically.
3	Send alerts	DOWN, RECOVERED and SLOW alerts appear on the dashboard and in the log, with optional Telegram, e-mail and webhook delivery.
4	Show performance reports	Per-endpoint uptime %, average, p95, minimum and maximum response time for 1 hour, 24 hours or 7 days.
5	Use cron jobs, logs and notifications	node-cron scheduler, file-based log, alert engine.
6	Real-time data handling	WebSocket push updates, live pop-up alerts and a LIVE indicator on the dashboard.
7	Persistent storage	SQLite database with four related tables and indexes.
8	Incident history	Each outage is stored with start time, end time, duration and cause.

3. Methodology / Implementation

3.1 System architecture

The system follows a client-server design. The server contains the scheduler, the health checker, the incident and alert logic, the database layer and a REST/WebSocket interface. The browser dashboard only displays data and sends commands (add, delete or check an endpoint). The components and their connections are shown in Figure 1, and the work-flow of a single check is described in Section 3.2.

3.2 Work-flow of a health check

- **Schedule.** A cron job runs every 5 seconds. For each endpoint it compares the time since the last check with the endpoint's interval (10 s to 5 min) and starts the check when it is due. A guard prevents two checks of the same endpoint from overlapping.
- **Request.** The checker sends an HTTP GET with an 8-second timeout and measures the elapsed time. A response with a status outside 200-299, a timeout or a network error counts as a failure.
- **Store.** The result (time stamp, success flag, status code, response time, error text) is inserted into the `checks` table.

- **Decide.** A success resets the failure counter; a failure increments it. When the counter reaches 2 the endpoint is marked DOWN, an incident row is opened and a DOWN alert is raised. The first success after that closes the incident, computes the downtime and raises a RECOVERED alert. A success slower than 1000 ms raises a SLOW alert (limited to one per endpoint every 5 minutes to avoid flooding).
- **Notify.** The alert is stored in the `alerts` table, written to the log, broadcast to open dashboards over WebSocket and sent to any configured external channel.
- **Display.** The dashboard refreshes instantly on every WebSocket message (with a 30-second fallback poll) and renders cards, bars, the chart, tables and logs.

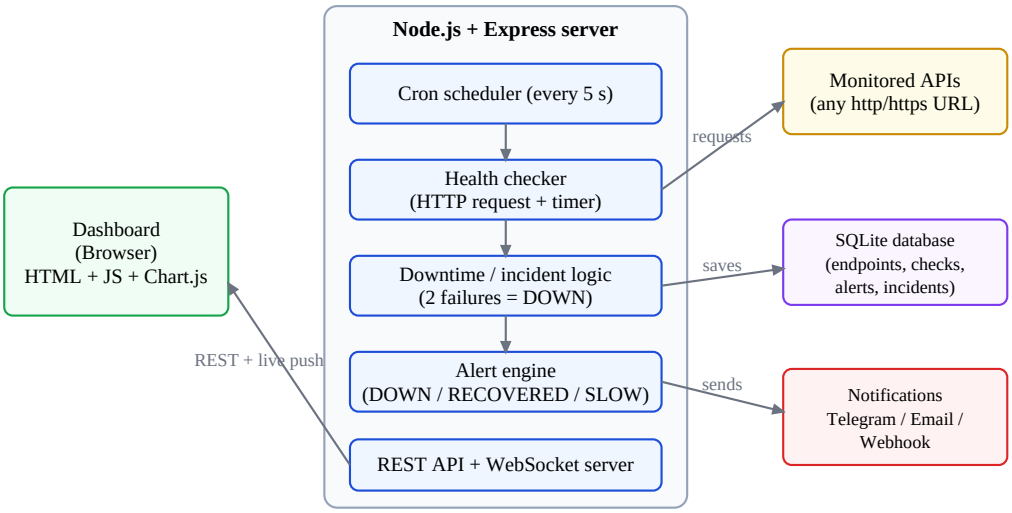


Figure 1: Architecture of the API Monitoring & Alert System

3.3 Why two failures before DOWN?

A single failed request can be caused by a momentary network glitch. Requiring two consecutive failures removes most false alarms while still detecting a real outage within about two check intervals.

3.4 Technology stack

Layer	Technology	Purpose
Runtime	Node.js 22+	Server-side JavaScript; built-in fetch and node:sqlite
Web framework	Express 5	REST API and static file hosting
Scheduler	node-cron	Cron expressions for the 5-second check loop and hourly clean-up
Database	SQLite (node:sqlite)	Embedded relational database, no installation required
Real time	ws (WebSocket)	Push updates and live alerts to the browser
E-mail	Nodemailer	Optional SMTP alert delivery
Frontend	HTML, CSS, JavaScript, Chart.js	Dashboard, charts and responsive layout

3.5 Database design

Table	Columns	Description
`endpoints`	id, name, url, interval_sec, created_at	The APIs being monitored
`checks`	id, endpoint_id, ts, ok, status, ms, error	One row per health check (indexed on endpoint_id, ts)
`alerts`	id, endpoint_id, name, type, message, ts	DOWN / RECOVERED / SLOW / TEST alerts
`incidents`	id, endpoint_id, started_at, ended_at, reason	Outages; ended_at is NULL while ongoing

The `checks` and `incidents` tables reference `endpoints` with ON DELETE CASCADE, so deleting an endpoint removes its history. Checks older than 7 days are pruned hourly.

3.6 REST API

Method	Route	Purpose
GET	/api/summary?hours=24	Status and performance report for every endpoint
GET	/api/endpoints/:id/checks	Recent check history (used by the chart)
POST	/api/endpoints	Add an endpoint {name, url, intervalSec}; only http/https URLs are accepted
DELETE	/api/endpoints/:id	Remove an endpoint and its history
POST	/api/endpoints/:id/check	Run a check immediately
GET	/api/alerts	Alert history
GET	/api/incidents	Incident history
POST	/api/alerts/test	Send a test alert through the configured channels
GET	/api/logs	Latest log lines
WS	/ws	Live updates pushed to the dashboard

3.7 Demo targets and configuration

To make the system demonstrable offline, the server exposes two built-in test endpoints: `/demo/healthy` (always responds) and `/demo/flaky`, which can be switched on and off with the **Toggle demo outage** button to simulate an outage. Notification channels are configured in an optional `.env` file (Telegram bot token and chat id, SMTP details, webhook URL); secrets are never committed to the repository.

4. Source Code and Technical Details

The complete source code is available in the GitHub repository: https://github.com/SinchanaRG26/sparkit_internship/tree/main/project1-api-monitor. The main files are listed below.

File	Role
`server.js`	Express server, cron scheduler, health checker, incident and alert logic, REST API, WebSocket
`db.js`	SQLite connection and table definitions
`public/index.html`	Dashboard (HTML, CSS and JavaScript)
`package.json`	Dependencies and the npm start script
`.env.example`	Template for notification settings

4.1 db.js

```
/**
 * Database layer (SQLite via Node built-in node:sqlite).
 * Tables: endpoints, checks, alerts, incidents
 */
const { DatabaseSync } = require('node:sqlite'); // built into Node 22.13+ (no install/compile needed)
const fs = require('fs');
const path = require('path');

const DATA_DIR = path.join(__dirname, 'data');
fs.mkdirSync(DATA_DIR, { recursive: true });

const db = new DatabaseSync(process.env.DB_FILE || path.join(DATA_DIR, 'monitor.db'));
db.exec('PRAGMA journal_mode = WAL; PRAGMA foreign_keys = ON;');

db.exec(`
CREATE TABLE IF NOT EXISTS endpoints (
  id           INTEGER PRIMARY KEY AUTOINCREMENT,
  name         TEXT    NOT NULL,
  url          TEXT    NOT NULL,
  interval_sec INTEGER NOT NULL DEFAULT 30,
  created_at   INTEGER NOT NULL
);

CREATE TABLE IF NOT EXISTS checks (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  endpoint_id INTEGER NOT NULL REFERENCES endpoints(id) ON DELETE CASCADE,
  ts          INTEGER NOT NULL,
  ok          INTEGER NOT NULL,
  status      INTEGER NOT NULL,
  ms          INTEGER NOT NULL,
  error       TEXT
);
CREATE INDEX IF NOT EXISTS idx_checks_ep_ts ON checks(endpoint_id, ts);

CREATE TABLE IF NOT EXISTS alerts (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  endpoint_id INTEGER,
  name        TEXT    NOT NULL,
  type        TEXT    NOT NULL,   -- DOWN | RECOVERED | SLOW | TEST
  message     TEXT    NOT NULL,
  ts          INTEGER NOT NULL
);
CREATE INDEX IF NOT EXISTS idx_alerts_ts ON alerts(ts);

CREATE TABLE IF NOT EXISTS incidents (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  endpoint_id INTEGER NOT NULL REFERENCES endpoints(id) ON DELETE CASCADE,
  started_at  INTEGER NOT NULL,
  ended_at    INTEGER,            -- NULL while the outage is ongoing
  reason      TEXT
);
CREATE INDEX IF NOT EXISTS idx_incidents_ep ON incidents(endpoint_id, started_at);
`);

module.exports = db;
```

4.2 server.js

```
/**
 * API Monitoring & Alert System - backend
 * Concepts: Cron jobs, Logs, Notifications, SQLite, WebSockets
 */
const fs = require('fs');
const path = require('path');

// ---------- tiny .env loader (so Windows users don't need to set env vars) ----------
try {
  const envFile = path.join(__dirname, '.env');
  if (fs.existsSync(envFile)) {
    for (const line of fs.readFileSync(envFile, 'utf8').split(/\r?\n/)) {
      const m = line.match(/^\s*([A-Z0-9_]+)\s*=\s*(.*?)\s*$/i);
      if (m && !line.trim().startsWith('#') && process.env[m[1]] === undefined) {
        process.env[m[1]] = m[2].replace(/^["']|["']$/g, '');
      }
    }
  }
} catch (e) { console.error('.env read failed:', e.message); }

const http = require('http');
const express = require('express');
const cron = require('node-cron');
const nodemailer = require('nodemailer');
const { WebSocketServer } = require('ws');
const db = require('./db');

const PORT = process.env.PORT || 3000;
const LOG_FILE = path.join(__dirname, 'data', 'monitor.log');
const FAILURE_THRESHOLD = 2;   // consecutive failures before DOWN
const SLOW_MS = 1000;          // slower than this = SLOW alert
const SLOW_ALERT_COOLDOWN_MS = 5 * 60 * 1000; // don't spam SLOW alerts
const KEEP_CHECKS_DAYS = 7;

// ---------- logging ----------
function log(level, msg) {
  const line = `${new Date().toISOString()} [${level}] ${msg}`;
  console.log(line);
  fs.appendFile(LOG_FILE, line + '\n', () => {});
}

// ---------- prepared statements ----------
const q = {
  endpoints: db.prepare('SELECT id, name, url, interval_sec AS intervalSec, created_at AS createdAt FROM
      endpoints ORDER BY id'),
  endpoint: db.prepare('SELECT id, name, url, interval_sec AS intervalSec FROM endpoints WHERE id = ?'),
  addEndpoint: db.prepare('INSERT INTO endpoints (name, url, interval_sec, created_at) VALUES (?, ?, ?, ?)'),
  delEndpoint: db.prepare('DELETE FROM endpoints WHERE id = ?'),
  addCheck: db.prepare('INSERT INTO checks (endpoint_id, ts, ok, status, ms, error) VALUES (?, ?, ?, ?, ?,
      ?)'),
  checksSince: db.prepare('SELECT ts, ok, status, ms, error FROM checks WHERE endpoint_id = ? AND ts >= ?
      ORDER BY ts'),
  recentChecks: db.prepare('SELECT ts, ok, status, ms, error FROM (SELECT * FROM checks WHERE endpoint_id = ?
       ORDER BY ts DESC LIMIT ?) ORDER BY ts'),
  lastCheck: db.prepare('SELECT ts, ok, status, ms, error FROM checks WHERE endpoint_id = ? ORDER BY ts DESC
      LIMIT 1'),
  addAlert: db.prepare('INSERT INTO alerts (endpoint_id, name, type, message, ts) VALUES (?, ?, ?, ?, ?)'),
  alerts: db.prepare('SELECT id, endpoint_id AS endpointId, name, type, message, ts FROM alerts ORDER BY ts
      DESC LIMIT ?'),
  openIncident: db.prepare('SELECT id, started_at FROM incidents WHERE endpoint_id = ? AND ended_at IS NULL
      ORDER BY id DESC LIMIT 1'),
  startIncident: db.prepare('INSERT INTO incidents (endpoint_id, started_at, reason) VALUES (?, ?, ?)'),
  endIncident: db.prepare('UPDATE incidents SET ended_at = ? WHERE id = ?'),
  incidents: db.prepare(`SELECT i.id, i.endpoint_id AS endpointId, e.name, i.started_at AS startedAt,
      i.ended_at AS endedAt, i.reason
                         FROM incidents i JOIN endpoints e ON e.id = i.endpoint_id ORDER BY i.started_at DESC
                              LIMIT ?`),
  prune: db.prepare('DELETE FROM checks WHERE ts < ?'),
  count: db.prepare('SELECT COUNT(*) AS n FROM endpoints'),
};

// ---------- seed demo endpoints on first run ----------
if (q.count.get().n === 0) {
  const now = Date.now();
  q.addEndpoint.run('Demo API (healthy)', `http://localhost:${PORT}/demo/healthy`, 30, now);
  q.addEndpoint.run('Demo API (flaky)', `http://localhost:${PORT}/demo/flaky`, 30, now);
  q.addEndpoint.run('JSONPlaceholder', 'https://jsonplaceholder.typicode.com/todos/1', 30, now);
}

// ---------- in-memory runtime state (rebuilt from DB on start) ----------
const state = new Map(); // endpointId -> { consecutiveFailures, isDown, lastCheck, lastSlowAlert }
function getState(id) {
  if (!state.has(id)) {
```

```
    const last = q.lastCheck.get(id);
    state.set(id, {
      consecutiveFailures: 0,
      isDown: !!q.openIncident.get(id),
      lastCheck: last ? last.ts : 0,
      lastSlowAlert: 0,
    });
  }
  return state.get(id);
}

// ---------- WebSocket (live updates) ----------
const app = express();
const server = http.createServer(app);
const wss = new WebSocketServer({ server, path: '/ws' });
function broadcast(type, payload = {}) {
  const msg = JSON.stringify({ type, ...payload });
  for (const c of wss.clients) if (c.readyState === 1) c.send(msg);
}

// ---------- notifications ----------
const mailer = process.env.SMTP_HOST
  ? nodemailer.createTransport({
      host: process.env.SMTP_HOST,
      port: +(process.env.SMTP_PORT || 587),
      secure: +(process.env.SMTP_PORT || 587) === 465,
      auth: { user: process.env.SMTP_USER, pass: process.env.SMTP_PASS },
    })
  : null;

function channels() {
  return {
    dashboard: true,
    email: !!(mailer && process.env.ALERT_EMAIL_TO),
    telegram: !!(process.env.TELEGRAM_BOT_TOKEN && process.env.TELEGRAM_CHAT_ID),
    webhook: !!process.env.WEBHOOK_URL,
  };
}

const ICON = { DOWN: '■', RECOVERED: '■', SLOW: '■', TEST: '■' };

async function sendExternal(type, name, message) {
  const text = `${ICON[type] || ''} [${type}] ${name}\n${message}`;
  const jobs = [];
  if (channels().email) {
    jobs.push(mailer.sendMail({
      from: process.env.SMTP_USER, to: process.env.ALERT_EMAIL_TO,
      subject: `API Monitor: ${type} - ${name}`, text,
    }).then(() => 'email'));
  }
  if (channels().webhook) {
    jobs.push(fetch(process.env.WEBHOOK_URL, {
      method: 'POST', headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ text, content: text }),
    }).then(r => { if (!r.ok) throw new Error('HTTP ' + r.status); return 'webhook'; }));
  }
  if (channels().telegram) {
    jobs.push(fetch(`https://api.telegram.org/bot${process.env.TELEGRAM_BOT_TOKEN}/sendMessage`, {
      method: 'POST', headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ chat_id: process.env.TELEGRAM_CHAT_ID, text }),
    }).then(async r => { if (!r.ok) throw new Error('HTTP ' + r.status + ' ' + (await r.text()).slice(0,
        120)); return 'telegram'; }));
  }
  const results = await Promise.allSettled(jobs);
  const sent = [], failed = [];
  results.forEach(r => (r.status === 'fulfilled' ? sent.push(r.value) : failed.push(r.reason.message)));
  failed.forEach(f => log('ERROR', 'Notification failed: ' + f));
  return { sent, failed };
}

async function notify(endpoint, type, message) {
  q.addAlert.run(endpoint.id ?? null, endpoint.name, type, message, Date.now());
  log(type === 'DOWN' ? 'ALERT' : 'INFO', `${endpoint.name}: [${type}] ${message}`);
  broadcast('alert', { alertType: type, name: endpoint.name, message });
  return sendExternal(type, endpoint.name, message);
}

// ---------- the checker ----------
async function checkEndpoint(ep) {
  const start = Date.now();
  let result;
  try {
    const res = await fetch(ep.url, { signal: AbortSignal.timeout(8000), headers: { 'User-Agent': 'api-
        monitor/1.0' } });
    result = { ts: start, ok: res.ok, status: res.status, ms: Date.now() - start, error: res.ok ? null :
        `HTTP ${res.status}` };
```

```
  } catch (e) {
    result = { ts: start, ok: false, status: 0, ms: Date.now() - start, error: e.name === 'TimeoutError' ?
        'Timeout (8s)' : (e.cause?.code || e.message) };
  }

  // The endpoint may have been deleted while the request was in flight
  if (!q.endpoint.get(ep.id)) return result;
  q.addCheck.run(ep.id, result.ts, result.ok ? 1 : 0, result.status, result.ms, result.error);

  const st = getState(ep.id);
  st.lastCheck = result.ts;

  if (result.ok) {
    if (st.isDown) {
      st.isDown = false;
      const inc = q.openIncident.get(ep.id);
      let duration = '';
      if (inc) { q.endIncident.run(Date.now(), inc.id); duration = ` after ${fmtDuration(Date.now() -
          inc.started_at)} of downtime`; }
      st.consecutiveFailures = 0;
      await notify(ep, 'RECOVERED', `Back online (${result.ms}ms)${duration}`);
    } else if (result.ms > SLOW_MS && Date.now() - st.lastSlowAlert > SLOW_ALERT_COOLDOWN_MS) {
      st.lastSlowAlert = Date.now();
      await notify(ep, 'SLOW', `Slow response: ${result.ms}ms (threshold ${SLOW_MS}ms)`);
    }
    st.consecutiveFailures = 0;
  } else {
    st.consecutiveFailures++;
    log('WARN', `${ep.name} check failed (${st.consecutiveFailures}/${FAILURE_THRESHOLD}): ${result.error}`);
    if (!st.isDown && st.consecutiveFailures >= FAILURE_THRESHOLD) {
      st.isDown = true;
      q.startIncident.run(ep.id, Date.now(), result.error);
      await notify(ep, 'DOWN', `Downtime detected: ${result.error}`);
    }
  }
  broadcast('check', { endpointId: ep.id });
  return result;
}

function fmtDuration(ms) {
  const s = Math.round(ms / 1000);
  if (s < 60) return `${s}s`;
  const m = Math.floor(s / 60);
  return m < 60 ? `${m}m ${s % 60}s` : `${Math.floor(m / 60)}h ${m % 60}m`;
}

// Cron: every 5 seconds, run checks for endpoints whose interval has elapsed
const running = new Set();
cron.schedule('*/5 * * * * *', () => {
  const now = Date.now();
  for (const ep of q.endpoints.all()) {
    if (running.has(ep.id)) continue;
    if (now - getState(ep.id).lastCheck >= ep.intervalSec * 1000 - 500) {
      running.add(ep.id);
      checkEndpoint(ep).catch(e => log('ERROR', e.message)).finally(() => running.delete(ep.id));
    }
  }
});
// Cron: hourly cleanup of old check rows
cron.schedule('0 * * * *', () => {
  const r = q.prune.run(Date.now() - KEEP_CHECKS_DAYS * 86400000);
  if (r.changes) log('INFO', `Pruned ${r.changes} old checks`);
});

// ---------- reports ----------
function percentile(sorted, p) {
  return sorted.length ? sorted[Math.min(sorted.length - 1, Math.floor((p / 100) * sorted.length))] : null;
}

function summarize(ep, sinceMs) {
  const checks = q.checksSince.all(ep.id, Date.now() - sinceMs);
  const okChecks = checks.filter(c => c.ok);
  const times = okChecks.map(c => c.ms).sort((a, b) => a - b);
  const last = q.lastCheck.get(ep.id);
  const st = getState(ep.id);
  return {
    id: ep.id, name: ep.name, url: ep.url, intervalSec: ep.intervalSec,
    status: !last ? 'PENDING' : st.isDown ? 'DOWN' : 'UP',
    totalChecks: checks.length,
    failedChecks: checks.length - okChecks.length,
    uptimePct: checks.length ? +((okChecks.length / checks.length) * 100).toFixed(2) : null,
    avgMs: times.length ? Math.round(times.reduce((a, b) => a + b, 0) / times.length) : null,
    minMs: times[0] ?? null,
    maxMs: times.length ? times[times.length - 1] : null,
    p95Ms: percentile(times, 95),
    lastCheck: last ? { ...last, ok: !!last.ok } : null,
```

```
    recent: q.recentChecks.all(ep.id, 40).map(c => ({ ok: !!c.ok, ms: c.ms, ts: c.ts })),
  };
}

// ---------- web server ----------
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

// Built-in demo targets so the dashboard has something to monitor offline
let flakyDown = false;
app.get('/demo/healthy', (req, res) => setTimeout(() => res.json({ ok: true }), 40 + Math.random() * 160));
app.get('/demo/flaky', (req, res) => {
  if (flakyDown) return res.status(503).json({ error: 'Service Unavailable' });
  setTimeout(() => res.json({ ok: true }), 60 + Math.random() * 500);
});
app.get('/demo/flaky/state', (req, res) => res.json({ down: flakyDown }));
app.post('/demo/flaky/toggle', (req, res) => {
  flakyDown = typeof req.body?.down === 'boolean' ? req.body.down : !flakyDown;
  log('INFO', `Demo flaky API is now ${flakyDown ? 'DOWN' : 'UP'}`);
  res.json({ down: flakyDown });
});

app.get('/api/summary', (req, res) => {
  const hours = Math.max(1, Math.min(168, +req.query.hours || 24));
  res.json({ windowHours: hours, generatedAt: Date.now(), endpoints: q.endpoints.all().map(ep =>
      summarize(ep, hours * 3600000)) });
});

app.get('/api/endpoints/:id/checks', (req, res) => {
  res.json(q.recentChecks.all(+req.params.id, Math.min(500, +req.query.limit || 60)).map(c => ({ ...c, ok:
      !!c.ok })));
});

app.post('/api/endpoints', (req, res) => {
  const { name, url, intervalSec } = req.body || {};
  let parsed;
  try { parsed = new URL(url); } catch { return res.status(400).json({ error: 'A valid URL (with http/https)
      is required' }); }
  if (!/^https?:$/.test(parsed.protocol)) return res.status(400).json({ error: 'Only http/https URLs are
      allowed' });
  const interval = Math.max(10, Math.min(3600, +intervalSec || 30));
  const info = q.addEndpoint.run(String(name || url).slice(0, 60), url, interval, Date.now());
  const ep = q.endpoint.get(info.lastInsertRowid);
  checkEndpoint(ep).catch(() => {});
  broadcast('check', { endpointId: ep.id });
  res.status(201).json(ep);
});

app.delete('/api/endpoints/:id', (req, res) => {
  const id = +req.params.id;
  q.delEndpoint.run(id); // checks + incidents removed by ON DELETE CASCADE
  state.delete(id);
  broadcast('check', { endpointId: id });
  res.json({ ok: true });
});

app.post('/api/endpoints/:id/check', async (req, res) => {
  const ep = q.endpoint.get(+req.params.id);
  if (!ep) return res.status(404).json({ error: 'Not found' });
  res.json(await checkEndpoint(ep));
});

app.get('/api/alerts', (req, res) => res.json(q.alerts.all(Math.min(200, +req.query.limit || 50))));
app.get('/api/incidents', (req, res) => res.json(q.incidents.all(Math.min(200, +req.query.limit || 30))));
app.get('/api/channels', (req, res) => res.json(channels()));

app.post('/api/alerts/test', async (req, res) => {
  const r = await notify({ id: null, name: 'API Monitor' }, 'TEST', 'This is a test alert. Your notification
      channel works!');
  res.json({ channels: channels(), ...r });
});

app.get('/api/logs', (req, res) => {
  fs.readFile(LOG_FILE, 'utf8', (err, data) => {
    if (err) return res.json([]);
    res.json(data.trim().split('\n').slice(-(Math.min(300, +req.query.limit || 100))).reverse());
  });
});

server.listen(PORT, () => {
  log('INFO', `API Monitor running on http://localhost:${PORT}`);
  const c = channels();
  log('INFO', `Alert channels: dashboard${c.email ? ', email' : ''}${c.telegram ? ', telegram' :
      ''}${c.webhook ? ', webhook' : ''}`);
});
process.on('SIGINT', () => { db.close(); process.exit(0); });
```

4.3 public/index.html

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>API Monitor</title>
<script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.1/dist/chart.umd.min.js"></script>
<style>
  :root {
    --bg:#f6f7f9; --card:#fff; --text:#16181d; --muted:#6b7280; --border:#e5e7eb;
    --up:#16a34a; --down:#dc2626; --warn:#d97706; --accent:#2563eb;
  }
  @media (prefers-color-scheme: dark) {
    :root { --bg:#0e1116; --card:#171b22; --text:#e8eaed; --muted:#8b93a1; --border:#262c36;
        --accent:#5b9dff; }
  }
  * { box-sizing:border-box; }
  body { margin:0; font:14px/1.5 system-ui,-apple-system,Segoe UI,Roboto,sans-serif; background:var(--bg);
      color:var(--text); }
  header { display:flex; align-items:center; justify-content:space-between; gap:12px; flex-wrap:wrap;
      padding:14px 24px; border-bottom:1px solid var(--border); background:var(--card); position:sticky;
      top:0; z-index:5; }
  h1 { font-size:18px; margin:0; display:flex; align-items:center; gap:10px; }
  h2 { font-size:15px; margin:0 0 12px; }
  main { max-width:1200px; margin:0 auto; padding:20px 24px 60px; display:grid; gap:20px; }
  .card { background:var(--card); border:1px solid var(--border); border-radius:10px; padding:16px; }
  .grid { display:grid; grid-template-columns:repeat(auto-fill,minmax(270px,1fr)); gap:14px; }
  .tile { cursor:pointer; border:1px solid var(--border); border-radius:10px; padding:14px;
      background:var(--card); border-left:4px solid var(--muted); }
  .tile.selected { outline:2px solid var(--accent); }
  .tile.UP { border-left-color:var(--up); } .tile.DOWN { border-left-color:var(--down);
      background:rgba(220,38,38,.06); }
  .tile .name { font-weight:600; } .tile .url { color:var(--muted); font-size:12px; overflow:hidden; text-
      overflow:ellipsis; white-space:nowrap; }
  .badge { display:inline-block; padding:1px 8px; border-radius:99px; font-size:11px; font-weight:700;
      color:#fff; background:var(--muted); }
  .badge.UP,.badge.RECOVERED { background:var(--up); } .badge.DOWN { background:var(--down); } .badge.SLOW {
      background:var(--warn); } .badge.TEST { background:var(--accent); }
  .badge.ONGOING { background:var(--down); }
  .row { display:flex; justify-content:space-between; align-items:center; gap:8px; }
  .gap { display:flex; align-items:center; gap:8px; flex-wrap:wrap; }
  .big { font-size:22px; font-weight:700; margin:6px 0 2px; }
  .muted { color:var(--muted); font-size:12px; }
  button, input, select { font:inherit; padding:7px 12px; border-radius:7px; border:1px solid var(--border);
      background:var(--card); color:var(--text); }
  button { cursor:pointer; } button.primary { background:var(--accent); border-color:var(--accent);
      color:#fff; }
  button.danger { color:var(--down); }
  form { display:flex; gap:8px; flex-wrap:wrap; }
  form input[name=url] { flex:1 1 280px; }
  table { width:100%; border-collapse:collapse; } th,td { text-align:left; padding:8px 10px; border-
      bottom:1px solid var(--border); }
  th { color:var(--muted); font-weight:600; font-size:12px; }
  .two { display:grid; grid-template-columns:1fr 1fr; gap:20px; } @media (max-width:800px){ .two{grid-
      template-columns:1fr;} }
  .alert { padding:8px 0; border-bottom:1px solid var(--border); display:flex; gap:10px; align-
      items:baseline; }
  .logs { font:12px ui-monospace,Menlo,monospace; max-height:260px; overflow:auto; white-space:pre-wrap;
      color:var(--muted); }
  .scroll { max-height:300px; overflow:auto; }
  canvas { max-height:280px; }
  /* uptime bar */
  .bar { display:flex; gap:2px; margin-top:10px; height:22px; }
  .bar i { flex:1; border-radius:2px; background:var(--border); }
  .bar i.ok { background:var(--up); } .bar i.fail { background:var(--down); }
  /* live indicator */
  .live { font-size:12px; font-weight:600; display:inline-flex; align-items:center; gap:6px;
      color:var(--muted); }
  .live::before { content:""; width:8px; height:8px; border-radius:50%; background:var(--muted); }
  .live.on { color:var(--up); } .live.on::before { background:var(--up); animation:pulse 1.6s infinite; }
  @keyframes pulse { 0%{box-shadow:0 0 0 0 rgba(22,163,74,.5)} 70%{box-shadow:0 0 0 7px rgba(22,163,74,0)}
      100%{box-shadow:0 0 0 0 rgba(22,163,74,0)} }
  .chip { font-size:11px; padding:2px 8px; border-radius:99px; border:1px solid var(--border);
      color:var(--muted); }
  .chip.on { color:var(--up); border-color:var(--up); }
  #toast { position:fixed; right:20px; bottom:20px; max-width:340px; padding:12px 14px; border-radius:10px;
      color:#fff; background:var(--down); display:none; box-shadow:0 6px 24px rgba(0,0,0,.25); z-index:10; }
  #toast.RECOVERED { background:var(--up); } #toast.SLOW { background:var(--warn); } #toast.TEST {
      background:var(--accent); }
</style>
</head>
```

```
<body>
<header>
  <h1>API Monitor <span class="live" id="live">connecting…</span></h1>
  <div class="gap">
    <label class="muted">Report window
      <select id="hours"><option value="1">1h</option><option value="24" selected>24h</option><option
          value="168">7d</option></select>
    </label>
    <button id="toggleFlaky" title="Simulate an outage on the demo flaky API">Toggle demo outage</button>
  </div>
</header>

<main>
  <section class="card">
    <h2>Add an endpoint</h2>
    <form id="addForm">
      <input name="name" placeholder="Name (optional)">
      <input name="url" placeholder="https://api.example.com/health" required>
      <select name="intervalSec"><option value="10">every 10s</option><option value="30" selected>every
          30s</option><option value="60">every 1m</option><option value="300">every 5m</option></select>
      <button class="primary">Add</button>
    </form>
    <div class="muted" id="formMsg"></div>
  </section>

  <section><div class="grid" id="tiles"></div></section>

  <section class="card">
    <div class="row"><h2 id="chartTitle">Response time</h2><span class="muted">last 60 checks
        (ms)</span></div>
    <canvas id="chart"></canvas>
  </section>

  <section class="card">
    <h2>Performance report</h2>
    <div class="scroll"><table>
      <thead><tr><th>Endpoint</th><th>Status</th><th>Uptime</th><th>Avg</th><th>p95</th><th>Min /
          Max</th><th>Checks</th><th>Failed</th><th></th></tr></thead>
      <tbody id="report"></tbody>
    </table></div>
  </section>

  <section class="card">
    <h2>Incident history</h2>
    <div class="scroll"><table>
      <thead><tr><th>Endpoint</th><th>Started</th><th>Ended</th><th>Duration</th><th>Cause</th><th></th></tr>
          </thead>
      <tbody id="incidents"></tbody>
    </table></div>
  </section>

  <div class="two">
    <section class="card">
      <div class="row"><h2>Alerts</h2><button id="testAlert">Send test alert</button></div>
      <div class="gap" id="channels" style="margin-bottom:10px"></div>
      <div class="scroll" id="alerts"></div>
    </section>
    <section class="card"><h2>Logs</h2><div class="logs" id="logs"></div></section>
  </div>
</main>
<div id="toast"></div>

<script>
let selected = null, chart = null, summary = null;
const $ = s => document.querySelector(s);
const esc = s => String(s ?? '').replace(/[&<>"']/g, c =>
    ({'&':'&amp;','<':'&lt;','>':'&gt;','"':'&quot;',"'":'&#39;'}[c]));
const api = (p, o) => fetch(p, o).then(r => r.json());
const fmtTime = ts => new Date(ts).toLocaleTimeString();
const ms = v => v == null ? '–' : v + ' ms';
function dur(a, b) {
  const s = Math.round(((b ?? Date.now()) - a) / 1000);
  if (s < 60) return s + 's';
  const m = Math.floor(s / 60);
  return m < 60 ? `${m}m ${s % 60}s` : `${Math.floor(m / 60)}h ${m % 60}m`;
}

async function refresh() {
  summary = await api('/api/summary?hours=' + $('#hours').value);
  if (selected == null || !summary.endpoints.find(e => e.id === selected)) selected =
      summary.endpoints[0]?.id ?? null;
  renderTiles(); renderReport();
  await Promise.all([renderChart(), renderAlerts(), renderIncidents(), renderLogs()]);
}

function renderTiles() {
```

```
  $('#tiles').innerHTML = summary.endpoints.map(e => `
    <div class="tile ${e.status} ${e.id === selected ? 'selected' : ''}" data-id="${e.id}">
      <div class="row"><span class="name">${esc(e.name)}</span><span class="badge
          ${e.status}">${e.status}</span></div>
      <div class="url">${esc(e.url)}</div>
      <div class="big">${ms(e.lastCheck?.ms)}</div>
      <div class="muted">${e.uptimePct == null ? 'No data yet' : e.uptimePct + '% uptime · avg ' +
          ms(e.avgMs)}</div>
      <div class="bar" title="Last ${e.recent.length} checks (green = up, red = failed)">
        ${e.recent.map(c => `<i class="${c.ok ? 'ok' : 'fail'}" title="${fmtTime(c.ts)} · ${c.ok ? c.ms + '
            ms' : 'failed'}"></i>`).join('')}
      </div>
    </div>`).join('') || '<div class="muted">No endpoints yet. Add one above.</div>';
  document.querySelectorAll('.tile').forEach(t => t.onclick = () => { selected = +t.dataset.id;
      renderTiles(); renderChart(); });
}

function renderReport() {
  $('#report').innerHTML = summary.endpoints.map(e => `
    <tr>
      <td><b>${esc(e.name)}</b></td>
      <td><span class="badge ${e.status}">${e.status}</span></td>
      <td>${e.uptimePct == null ? '–' : e.uptimePct + '%'}</td>
      <td>${ms(e.avgMs)}</td><td>${ms(e.p95Ms)}</td>
      <td>${e.minMs ?? '–'} / ${e.maxMs ?? '–'}</td>
      <td>${e.totalChecks}</td><td>${e.failedChecks}</td>
      <td><button data-check="${e.id}">Check now</button> <button class="danger" data-
          del="${e.id}">Delete</button></td>
    </tr>`).join('');
  document.querySelectorAll('[data-check]').forEach(b => b.onclick = async () => { await
      api(`/api/endpoints/${b.dataset.check}/check`, { method: 'POST' }); });
  document.querySelectorAll('[data-del]').forEach(b => b.onclick = async () => {
    if (confirm('Delete this endpoint and its history?')) await api(`/api/endpoints/${b.dataset.del}`, {
        method: 'DELETE' });
  });
}

async function renderChart() {
  if (selected == null) return;
  const ep = summary.endpoints.find(e => e.id === selected);
  $('#chartTitle').textContent = 'Response time – ' + (ep?.name || '');
  const checks = await api(`/api/endpoints/${selected}/checks?limit=60`);
  const data = {
    labels: checks.map(c => fmtTime(c.ts)),
    datasets: [
      { label: 'Response time (ms)', data: checks.map(c => c.ok ? c.ms : null), borderColor: '#2563eb',
          backgroundColor: 'rgba(37,99,235,.12)', fill: true, tension: .3, pointRadius: 2, spanGaps: false },
      { label: 'Failed check', data: checks.map(c => c.ok ? null : 0), borderColor: '#dc2626',
          backgroundColor: '#dc2626', showLine: false, pointRadius: 6, pointStyle: 'crossRot' },
    ],
  };
  if (chart) { chart.data = data; chart.update(); }
  else chart = new Chart($('#chart'), { type: 'line', data, options: { animation: false, responsive: true,
      interaction: { mode: 'index', intersect: false },
    scales: { y: { beginAtZero: true, title: { display: true, text: 'ms' } }, x: { ticks: { maxTicksLimit: 8
        } } } } });
}

async function renderIncidents() {
  const list = await api('/api/incidents?limit=30');
  $('#incidents').innerHTML = list.map(i => `
    <tr>
      <td><b>${esc(i.name)}</b></td>
      <td>${new Date(i.startedAt).toLocaleString()}</td>
      <td>${i.endedAt ? new Date(i.endedAt).toLocaleString() : '<span class="badge
          ONGOING">ONGOING</span>'}</td>
      <td>${dur(i.startedAt, i.endedAt)}</td>
      <td>${esc(i.reason || '')}</td><td></td>
    </tr>`).join('') || '<tr><td colspan="6" class="muted">No incidents recorded. Try “Toggle demo
        outage”.</td></tr>';
}

async function renderAlerts() {
  const [alerts, ch] = await Promise.all([api('/api/alerts?limit=30'), api('/api/channels')]);
  $('#channels').innerHTML = Object.entries(ch).map(([k, v]) => `<span class="chip ${v ? 'on' : ''}">${v ?
      '●' : '■'} ${k}</span>`).join('');
  $('#alerts').innerHTML = alerts.map(a => `
    <div class="alert"><span class="badge ${a.type}">${a.type}</span>
      <div><b>${esc(a.name)}</b> – ${esc(a.message)}<div class="muted">${new
          Date(a.ts).toLocaleString()}</div></div></div>`).join('')
    || '<div class="muted">No alerts yet. Try “Toggle demo outage”.</div>';
}

async function renderLogs() { $('#logs').textContent = (await api('/api/logs?limit=60')).join('\n'); }

function toast(type, text) {
```

```
  const t = $('#toast'); t.className = type; t.textContent = text; t.style.display = 'block';
  clearTimeout(toast.t); toast.t = setTimeout(() => t.style.display = 'none', 6000);
}

// ---- live updates over WebSocket (falls back to polling) ----
let refreshTimer = null;
function scheduleRefresh() { clearTimeout(refreshTimer); refreshTimer = setTimeout(refresh, 250); }
function connect() {
  const ws = new WebSocket((location.protocol === 'https:' ? 'wss://' : 'ws://') + location.host + '/ws');
  ws.onopen = () => { $('#live').textContent = 'LIVE'; $('#live').classList.add('on'); };
  ws.onclose = () => { $('#live').textContent = 'reconnecting…'; $('#live').classList.remove('on');
      setTimeout(connect, 2000); };
  ws.onmessage = ev => {
    const m = JSON.parse(ev.data);
    if (m.type === 'alert') toast(m.alertType, `${m.alertType}: ${m.name} – ${m.message}`);
    scheduleRefresh();
  };
}

$('#addForm').onsubmit = async e => {
  e.preventDefault();
  const f = Object.fromEntries(new FormData(e.target));
  const r = await fetch('/api/endpoints', { method: 'POST', headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(f) });
  const j = await r.json();
  $('#formMsg').textContent = r.ok ? 'Added.' : j.error;
  if (r.ok) e.target.reset();
};
$('#toggleFlaky').onclick = async () => {
  const r = await api('/demo/flaky/toggle', { method: 'POST', headers: { 'Content-Type': 'application/json'
      }, body: '{}' });
  setToggleLabel(r.down);
};
function setToggleLabel(down) { $('#toggleFlaky').textContent = down ? 'Restore demo API' : 'Toggle demo
    outage'; }
$('#testAlert').onclick = async () => {
  const r = await api('/api/alerts/test', { method: 'POST' });
  const parts = [];
  if (r.sent.length) parts.push('sent via ' + r.sent.join(', '));
  if (r.failed.length) parts.push('failed: ' + r.failed.join('; '));
  toast('TEST', 'Test alert ' + (parts.join(' | ') || 'shown on dashboard only (no email/Telegram
      configured)'));
};
$('#hours').onchange = refresh;

api('/demo/flaky/state').then(s => setToggleLabel(s.down));
connect();
refresh();
setInterval(refresh, 30000); // safety net if the WebSocket is down
</script>
</body>
</html>
```

5. Output Screenshots

The following screenshots were taken from the running application at http://localhost:3000.

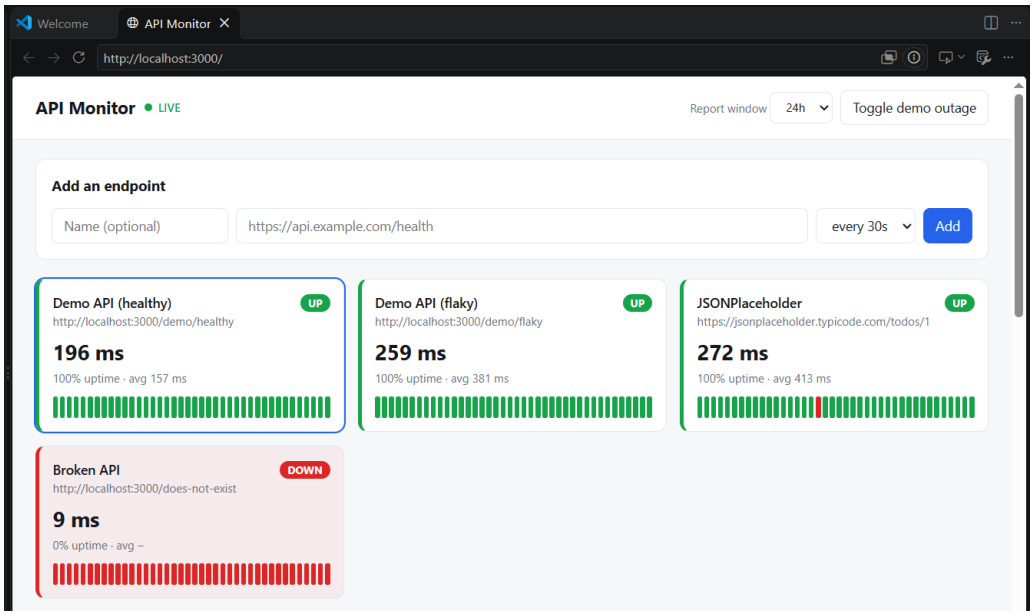


Figure 2: Dashboard status cards. Three endpoints are UP with green uptime bars; the Broken API endpoint (a non-existent URL returning HTTP 404) is DOWN and shown in red. The green LIVE badge shows the WebSocket connection is active.

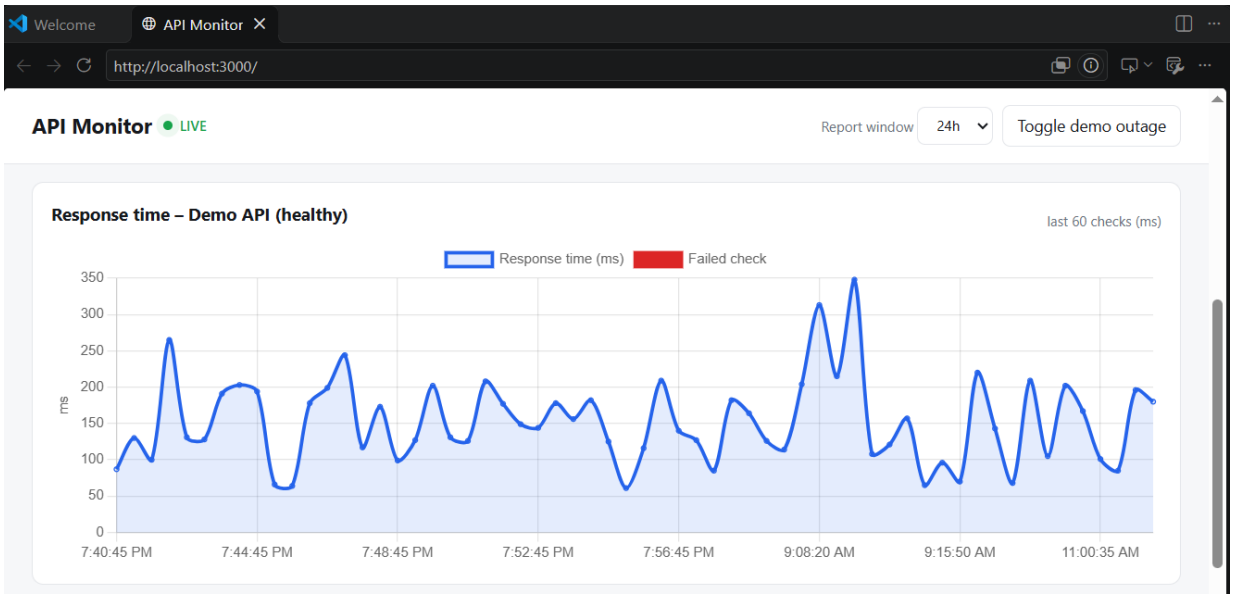


Figure 3: Response-time chart for the Demo API (healthy) over its last 60 checks.

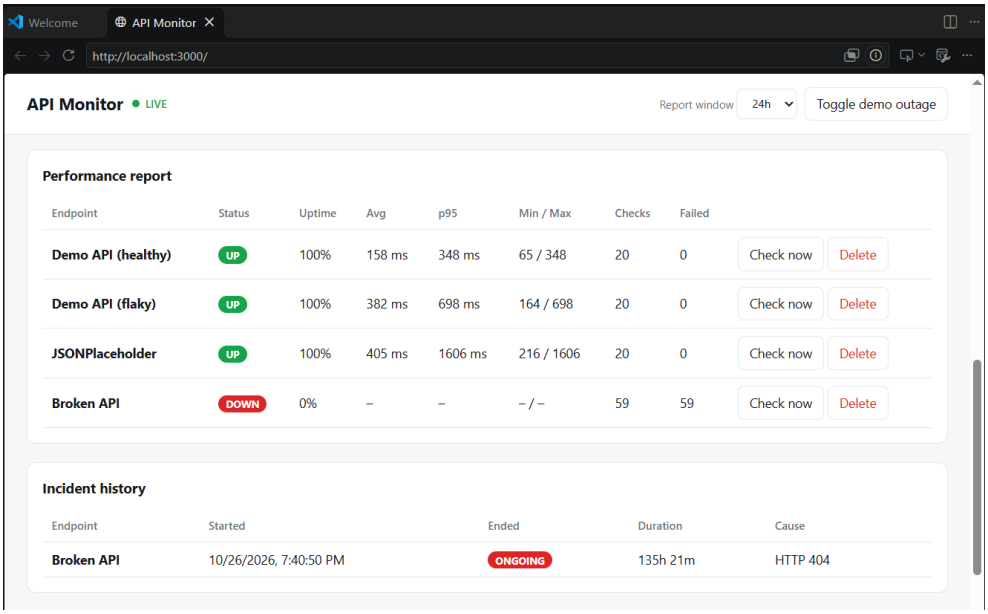


Figure 4: Performance report (uptime, average, p95, min/max, checks, failed) and the Incident history showing the ongoing Broken API outage.

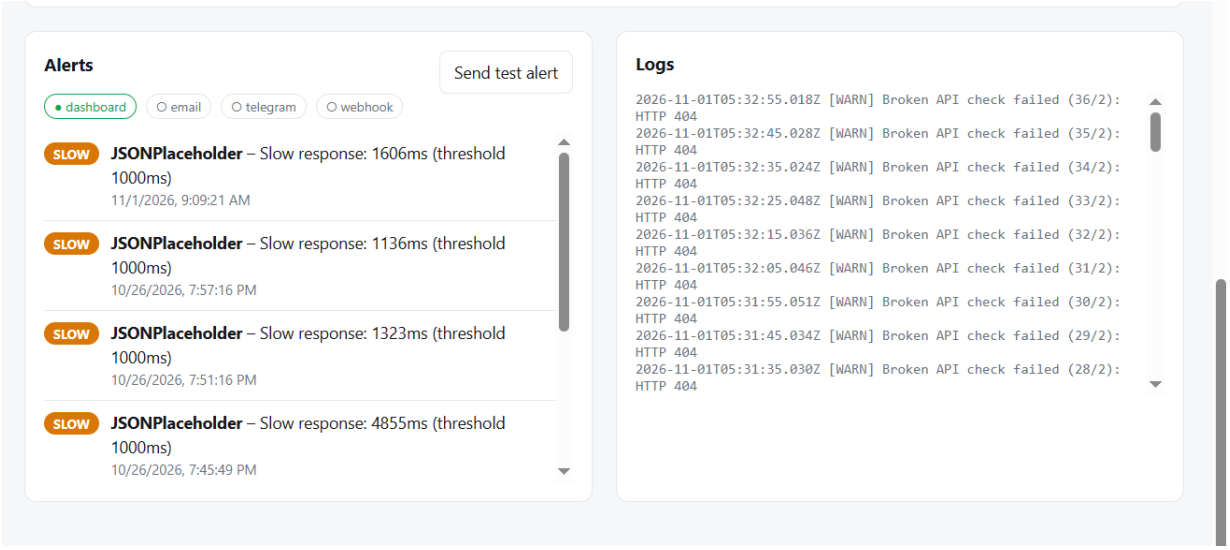


Figure 5: Alerts panel with SLOW alerts and notification-channel indicators, next to the live log panel showing failed checks.

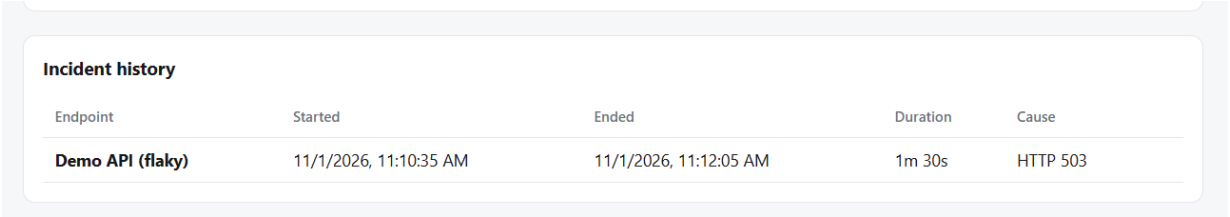


Figure 6: Incident history after the outage test. The Demo API (flaky) outage (cause HTTP 503) was detected and then closed automatically when the API recovered, with start time, end time and a duration of 1 minute 30 seconds.

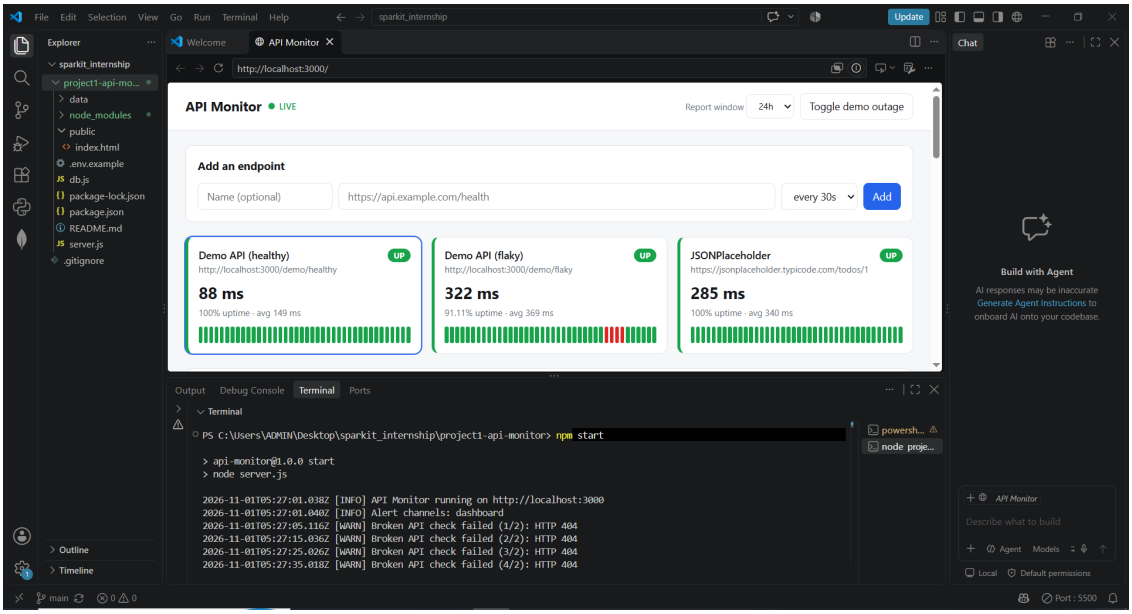


Figure 7: Project folder structure in VS Code (project1-api-monitor with server.js, db.js, public, .env.example and the repository .gitignore) and the server start-up in the terminal, with the dashboard running in the built-in browser. The Demo API (flaky) card shows 91.11% uptime and red blocks from the simulated outage.

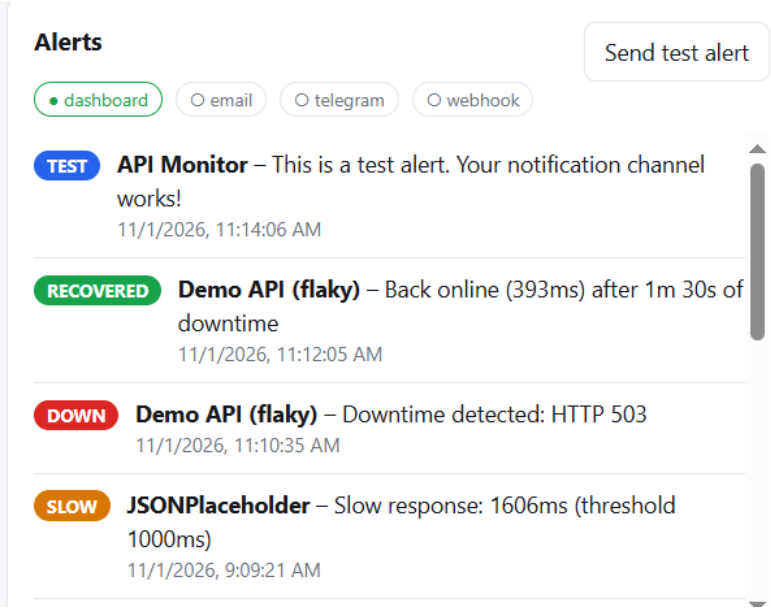


Figure 8: Alerts panel with the DOWN alert (Demo API (flaky), HTTP 503, 11:10:35), the RECOVERED alert (back online after 1m 30s of downtime, 11:12:05), a SLOW alert for JSONPlaceholder and a test alert. The channel indicators show the dashboard channel is active.

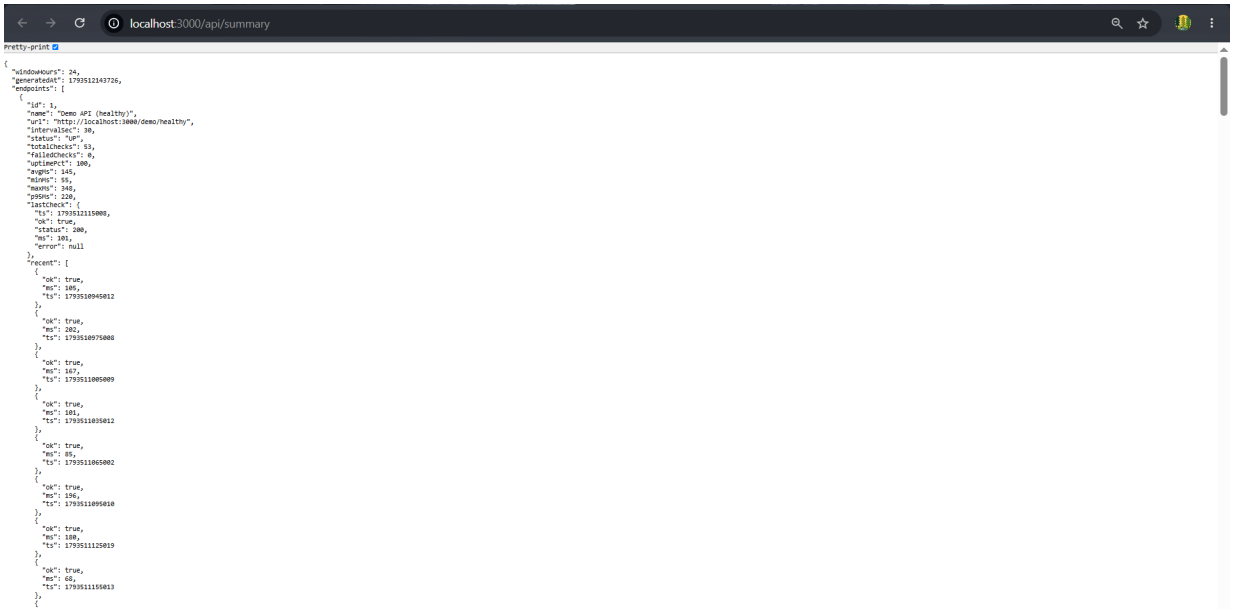


Figure 9: Raw JSON returned by the REST API (http://localhost:3000/api/summary), showing the status, uptime, response-time statistics and recent checks that the dashboard reads from the backend.

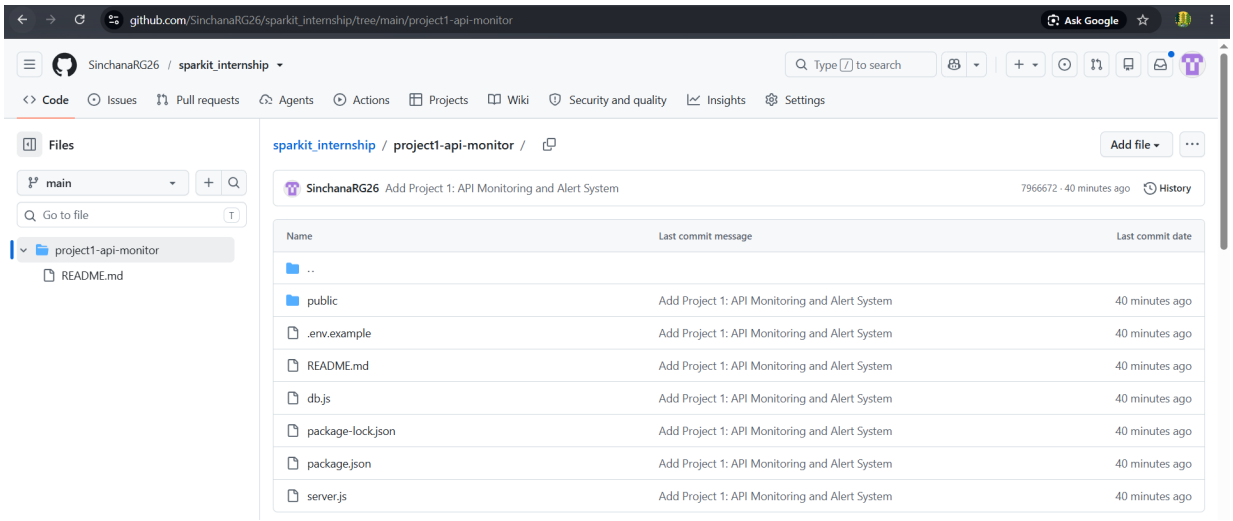
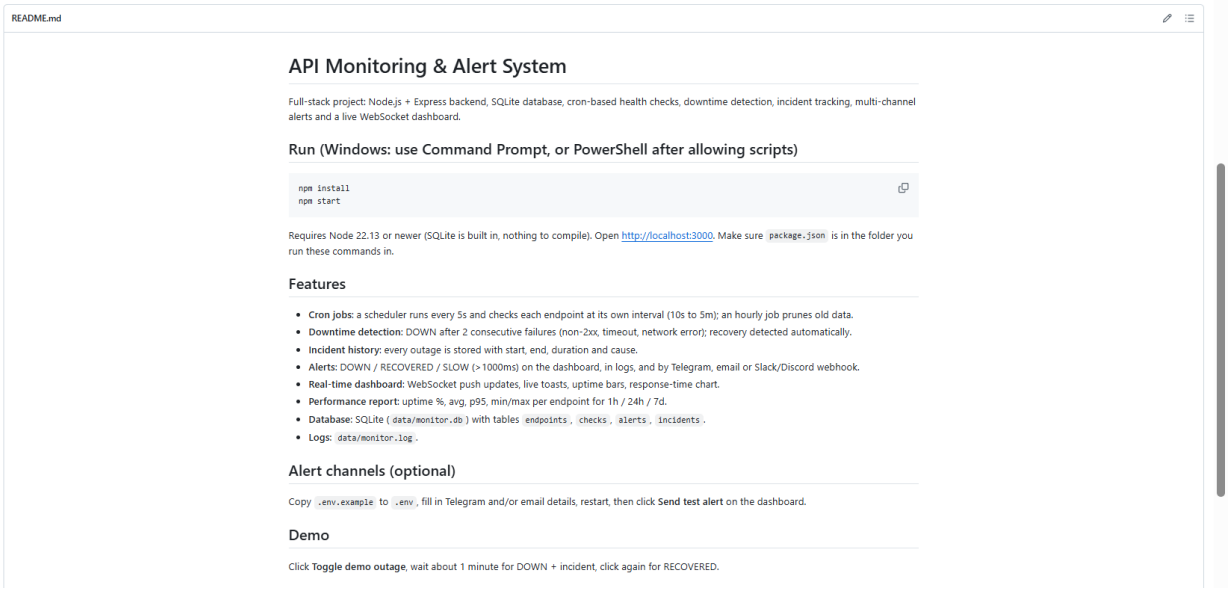


Figure 10: Project source code in the GitHub repository (sparkit_internship / project1-api-monitor). No .env file, database or node_modules is uploaded.


README.md

API Monitoring & Alert System

Full-stack project: Node.js + Express backend, SQLite database, cron-based health checks, downtime detection, incident tracking, multi-channel alerts and a live WebSocket dashboard.

Run (Windows: use Command Prompt, or PowerShell after allowing scripts)

```
npm install
npm start
```

Requires Node 22.13 or newer (SQLite is built in, nothing to compile). Open http://localhost:3000. Make sure `package.json` is in the folder you run these commands in.

Features

- **Cron jobs:** a scheduler runs every 5s and checks each endpoint at its own interval (10s to 5m); an hourly job prunes old data.
- **Downtime detection:** DOWN after 2 consecutive failures (non-2xx, timeout, network error); recovery detected automatically.
- **Incident history:** every outage is stored with start, end, duration and cause.
- **Alerts:** DOWN / RECOVERED / SLOW (>1000ms) on the dashboard, in logs, and by Telegram, email or Slack/Discord webhook.
- **Real-time dashboard:** WebSocket push updates, live toasts, uptime bars, response-time chart.
- **Performance report:** uptime %, avg, p95, min/max per endpoint for 1h / 24h / 7d.
- **Database:** SQLite (`data/monitor.db`) with tables `endpoints` , `checks` , `alerts` , `incidents` .
- **Logs:** `data/monitor.log` .

Alert channels (optional)

Copy `.env.example` to `.env` , fill in Telegram and/or email details, restart, then click **Send test alert** on the dashboard.

Demo

Click **Toggle demo outage**, wait about 1 minute for DOWN + incident, click again for RECOVERED.

Figure 11: The project README rendered on GitHub, with run instructions and the feature list.

6. Results and Analysis

6.1 Measured performance

The table below is taken from the Performance report (24-hour window) shown in Figure 4.

Endpoint	Status	Uptime	Avg	p95	Min / Max	Checks	Failed
Demo API (healthy)	UP	100%	158 ms	348 ms	65 / 348 ms	20	0
Demo API (flaky)	UP	100%	382 ms	698 ms	164 / 698 ms	20	0
JSONPlaceholder (public API)	UP	100%	405 ms	1606 ms	216 / 1606 ms	20	0
Broken API (/does-not-exist)	DOWN	0%	-	-	-	59	59

6.2 Test cases

Test	Expected behaviour	Result
Add a healthy endpoint	Card turns UP, response time and uptime bar appear	Passed (Figure 2)
Add an endpoint returning HTTP 404	Card turns DOWN after 2 failed checks; incident opened	Passed (Figures 2 and 4)
Slow response (> 1000 ms)	SLOW alert created and shown	Passed - SLOW alerts of 1136, 1323, 1606 and 4855 ms recorded (Figure 5)
Failed checks logged	Each failure written to the log with timestamp	Passed (Figure 5)

Test	Expected behaviour	Result
Endpoint recovers	RECOVERED alert and incident closed with duration	Passed - Demo API (flaky) outage of 1m 30s closed automatically (Figures 6, 7 and 8)
Invalid URL (e.g. file:// or ftp://)	Request rejected with an error	Passed - only http/https accepted
Delete an endpoint	Endpoint, checks and incidents removed	Passed (cascade delete)
REST API returns monitoring data	JSON with status, uptime and response-time statistics	Passed (Figure 9)
Dashboard updates without reload	New checks and alerts appear instantly	Passed - LIVE WebSocket indicator
External notification (Telegram, e-mail, webhook)	Message delivered when configured	Implemented and configurable; the test alert appears on the dashboard (Figure 8), external channels were not enabled in the demo run

6.3 Analysis

- **Latency differences.** The locally hosted demo endpoints answer fastest (average 158 ms and 382 ms, including the artificial delay built into the flaky endpoint), while the public JSONPlaceholder API averages 405 ms because the request crosses the internet.
- **Average versus p95.** For JSONPlaceholder the average is 405 ms but the 95th percentile is 1606 ms. A few slow responses hide behind a healthy-looking average, which is why p95 and maximum are reported as well. These slow checks are what triggered the SLOW alerts.
- **Downtime detection.** The Broken API endpoint failed all 59 of its checks (0% uptime). The system recorded the cause (HTTP 404), opened a single incident and kept it ONGOING instead of creating a new alert for every failed check, so the alert list is not flooded.
- **Scheduler behaviour.** The check interval stayed at about 10 seconds for the Broken API endpoint (see the log timestamps). The horizontal gap in the response-time chart (Figure 3) and the 'missed execution' notices from node-cron correspond to periods in which the host computer was suspended; the scheduler resumed automatically on wake-up. The Broken API incident duration of 135 hours 21 minutes therefore includes that period, since the incident stays open until a successful check.
- **Recovery test.** When the Demo API (flaky) was switched off with the Toggle demo outage button, the system opened an incident (started 11:10:35, cause HTTP 503) and raised a DOWN alert; when the API was restored, it raised a RECOVERED alert and closed the incident at 11:12:05 with a measured downtime of 1 minute 30 seconds (Figures 6 and 8). The endpoint's uptime dropped to 91.11% and its failed checks appear as red blocks in the uptime bar (Figure 7).
- **Persistence.** Because checks, alerts and incidents are stored in SQLite, the history survived server restarts, and an open incident was restored on start-up.

6.4 Limitations

- Checks run at fixed intervals (minimum 10 seconds); an outage shorter than one interval can be missed.
- A single server instance performs all checks from one location, so a network problem on the monitoring machine looks like an outage of the monitored API.
- There is no user login; all endpoints are visible to anyone who can open the dashboard.
- Only HTTP GET checks with a status-code test are supported; response-body validation and custom headers are not implemented.

7. Conclusion

The project delivers a working API monitoring service. It checks any http/https endpoint on a schedule, measures response time, stores the history in a relational database, detects downtime with a two-failure rule, tracks incidents from start to recovery, and raises DOWN, RECOVERED and SLOW alerts on the dashboard, in the logs and through optional Telegram, e-mail and webhook channels. The WebSocket-driven dashboard shows live status, uptime bars, a response-time chart and a performance report with uptime, average, p95 and minimum/maximum values.

Testing with healthy, slow and failing endpoints confirmed that the cron scheduler, logging and notification concepts work together as intended. Working on the project gave practical experience with scheduled background jobs, asynchronous HTTP requests and timeouts, relational schema design, REST API design, real-time communication with WebSockets and secure handling of configuration secrets.

Future enhancements: user authentication with per-user endpoints, response-body and header assertions, multi-region checks, a public status page, SMS or push notifications, exportable PDF/CSV reports, and a move to PostgreSQL for larger deployments.

8. References

- Node.js Documentation (fetch, node:sqlite) - https://nodejs.org/docs/latest/api/
- Express.js Guide - https://expressjs.com/ ; node-cron - https://github.com/node-cron/node-cron
- ws: a Node.js WebSocket library - https://github.com/websockets/ws ; SQLite - https://www.sqlite.org/docs.html
- Chart.js Documentation - https://www.chartjs.org/docs/ ; Nodemailer - https://nodemailer.com/
- Telegram Bot API - https://core.telegram.org/bots/api ; MDN Web Docs (Fetch, WebSocket) - https://developer.mozilla.org/
- RFC 9110: HTTP Semantics (status codes) - https://www.rfc-editor.org/rfc/rfc9110